BÁO CÁO DỰ ÁN CUỐI KỲ: SMARTDESK AI

HỆ THỐNG TRỢ LÝ HỖ TRỢ KHÁCH HÀNG THÔNG MINH & TỰ ĐỘNG HÓA TRI THỨC ĐA DOANH NGHIỆP

Môn học: Các Hệ thống AI Ứng dụng & Agentic Nâng cao (Advanced Agentic & Applied AI Systems)
Thời gian thực hiện: Học kỳ 6 (2026–2027)
Phiên bản tài liệu: 1.0.0 (Bản chính thức)
Phân quyền nhóm phát triển: - **Nhóm 1 / Dev A (AI Engine, RAG, Structured Output, Guardrails & Eval):** Nguyễn Nhật Huy (`@TheOrijinnn`) - **Nhóm 2 / Dev B & Integrator (Platform, Security, Database, Workspaces, Tickets & Shared Contracts):** Hoàng Văn Bắp (`@BapVitHoang`) - **Nhóm 3 / Dev C (Frontend UX, QA, Mocks & Capstone Documentation):** Trần Quốc Khánh (`@TKhanh`)

MỤC LỤC TỔNG QUAN

1. Chương 1: Giới thiệu bài toán, đối tượng người dùng & phạm vi hệ thống
2. Chương 2: 12 User Stories & Bảng Requirement Traceability Matrix (RTM)
3. Chương 3: Kiến trúc hệ thống & Architecture Decision Records (ADR-001..004)
4. Chương 4: Thiết kế AI chuyên sâu & Quy trình Đa Tác tử (Multi-Agent System)
5. Chương 5: An toàn Bảo mật, Hàng rào chắn (Guardrails), Quyền riêng tư & Dự phòng BM25
6. Chương 6: Kết quả thực nghiệm bộ 24 Test Cases, Chi phí Token & Độ trễ
7. Chương 7: Hạn chế hiện tại & Định hướng phát triển

CHƯƠNG 1: GIỚI THIỆU BÀI TOÁN, ĐỐI TƯỢNG NGƯỜI DÙNG & PHẠM VI HỆ THỐNG

1.1 Bối cảnh thực tế & Đặt vấn đề

Trong kỷ nguyên số, bộ phận Chăm sóc Khách hàng (Customer Support) của các doanh nghiệp vừa và lớn (SaaS, Thương mại điện tử, Dịch vụ y tế...) luôn phải đối mặt với áp lực khổng lồ: - **Tần suất lặp lại cao:** Hơn 60% yêu cầu gửi đến liên quan đến các chính sách phổ biến (chính sách hoàn tiền, quy trình đổi trả hàng, hướng dẫn bật xác thực hai bước 2FA, biểu phí dịch vụ). - **Thời gian phản hồi đầu tiên (First Response Time - FRT) kéo dài:** Trung bình từ 4 đến 12 giờ, gây giảm sút nghiêm trọng chỉ số hài lòng khách hàng (CSAT). - **Rủi ro ảo giác (Hallucination) từ mô hình ngôn ngữ lớn (LLM):** Khi ứng dụng Chatbot AI thông thường chưa được kiểm định chặt chẽ, mô hình thường tự ý bịa đặt chính sách bảo hành, cam kết sai thời hạn SLA hoặc tiết lộ thông tin bí mật kinh doanh. - **Gián đoạn dịch vụ khi quá tải API Cloud:** Các nền tảng chỉ phụ thuộc vào một nhà cung cấp AI đám mây (Cloud AI) dễ bị lỗi 429 (Too Many Requests / Resource Exhausted), khiến toàn bộ kênh hỗ trợ bị tê liệt.

1.2 Giải pháp SmartDesk AI

SmartDesk AI được thiết kế như một nền tảng hỗ trợ khách hàng thông minh và tự động hóa tri thức doanh nghiệp toàn diện. Hệ thống kết hợp: 1. **Kiến trúc RAG (Retrieval-Augmented Generation)** độ trễ thấp (< 1,500ms) kèm trích dẫn nguồn gốc minh bạch (Citation Badges). 2. **Quy trình Sát hạch Tri thức 3 Tác tử (3-Agent Active Knowledge Verification Studio):** Ngăn chặn tài liệu lỗi/mâu thuẫn được xuất bản vào cơ sở tri thức trước khi phục vụ khách hàng. 3. **AI Copilot hỗ trợ nhân viên CSKH:** Tự động phân loại danh mục, tính cam kết SLA chuẩn xác (`Urgent: 2h` , `High: 4h` , `Medium: 8h` , `Low: 24h`) và soạn trước bản thảo phản hồi có căn cứ. 4. **Dual AI Provider & Deterministic Fallback:** Hỗ trợ linh hoạt cả Cloud (Google Gemini 3.5 Flash Lite) và Local LLM (Ollama Qwen2.5 3B), đồng thời kích hoạt bộ so khớp phi AI Okapi BM25 cùng mạch ngắt Cost-Guard Circuit Breaker khi gặp sự cố mạng hoặc cạn kiệt ngân sách.

1.3 Đối tượng người dùng mục tiêu (Personas)

- **Khách hàng cuối (End-Customer / Role `user`):** Cần giải đáp thắc mắc 24/7 tức thì, xem nguồn trích dẫn đối chiếu, gửi phiếu hỗ trợ khi gặp sự cố phức tạp và theo dõi tiến độ phiếu của chính mình.
- **Nhân viên Hỗ trợ (Support Agent / Role `admin`):** Quản lý hàng đợi ticket, xem tóm tắt thông minh do AI phân tích, hiệu chỉnh bản thảo trước khi gửi khách hàng và đóng phiếu.
- **Quản trị viên Tri thức (Workspace Admin / Role `admin`):** Thiết lập Persona thương hiệu cho từng không gian làm việc (IT SaaS, Nha khoa SmileCare, Siêu thị Điện máy TechStore), nạp tài liệu PDF/Markdown và theo dõi giám sát chi phí token AI.

CHƯƠNG 2: 12 USER STORIES & BẢNG REQUIREMENT TRACEABILITY MATRIX (RTM)

2.1 Mười hai User Stories Chuẩn mực (`US-01` .. `US-12`)

Mỗi câu chuyện người dùng tuân thủ cấu trúc chuẩn **"As a ... I want to ... So that ..."** kèm Tiêu chí chấp nhận (Acceptance Criteria):

1. `US-01` **(Customer Grounded RAG Chat):** *Là một* khách hàng, *tôi muốn* đặt câu hỏi bằng ngôn ngữ tự nhiên và nhận câu trả lời có trích dẫn tài liệu cụ thể, *để* tôi tự giải quyết vấn đề mà không phải chờ đợi nhân viên.
2. *Acceptance:* Phản hồi kèm citation (`title` , `page` , `snippet`); nhiệt độ LLM `0.2` .
3. `US-02` **(Deterministic BM25 Fallback):** *Là một* người dùng khi hệ thống AI gặp sự cố mạng hoặc chạm giới hạn 429, *tôi muốn* hệ thống tự động trả lời bằng bộ so khớp FAQ BM25 và cung cấp nút chuyển tiếp sang ticket, *để* luồng hỗ trợ không bao giờ rơi vào ngõ cụt.
4. *Acceptance:* Chuyển đổi dưới 4.0s; gắn cờ `is_fallback: true` ; gợi ý form ticket.
5. `US-03` **(Bilingual Guardrails & Prompt Security):** *Là một* quản trị viên hệ thống, *tôi muốn* mọi truy vấn chứa mã tấn công Prompt Injection (cả Tiếng Anh và Tiếng Việt) bị chặn ngay tại tầng gateway, *để* bảo vệ an toàn chỉ thị hệ thống và dữ liệu doanh nghiệp.
6. *Acceptance:* Chặn đứng với HTTP 400 `PROMPT_SECURITY_VIOLATION` ; không gửi payload độc hại lên LLM.

7. `US-04` **(Ticket Submission & SLA Engine):** *Là một* khách hàng có sự cố chưa giải quyết được, *tôi muốn* tạo phiếu hỗ trợ với mã `#TICK-XXXX` và thời hạn cam kết SLA rõ ràng, *để* tôi biết khi nào chuyên viên sẽ phản hồi.
8. *Acceptance:* Định dạng `#TICK-XXXX`; SLA: Urgent (2h), High (4h), Medium (8h), Low (24h).
9. `US-05` **(AI Copilot Ticket Triage):** *Là một* nhân viên CSKH, *tôi muốn* các ticket mới được AI tự động phân loại nhãn và tạo sẵn bản thảo câu trả lời (Draft Reply), *để* tôi tiết kiệm thời gian gõ văn bản và phản hồi khách hàng trong vài giây.
10. *Acceptance:* Tạo nhãn `ai_tags` và `ai_draft_reply`; chỉ gửi sau khi có nhân viên duyệt.
11. `US-06` **(Agent Queue Sorting & Pagination):** *Là một* nhân viên CSKH, *tôi muốn* tìm kiếm, lọc theo danh mục, sắp xếp theo độ ưu tiên/SLA và phân trang danh sách ticket, *để* tôi không bỏ sót các sự cố khẩn cấp.
12. *Acceptance:* Dropdown sort đa tiêu chí; phân trang client-server; hỗ trợ xóa ticket (Admin).
13. `US-07` **(Multi-Workspace Isolation):** *Là một* quản trị viên công ty, *tôi muốn* tạo các Workspace độc lập với giọng điệu (Tone of Voice) và dữ liệu tài liệu riêng biệt, *để* kiến thức của tenant này không bị rò rỉ sang tenant khác.
14. *Acceptance:* Lọc triệt để theo `workspace_id`; 0% rò rỉ chéo tri thức giữa các Workspace.
15. `US-08` **(Document Ingestion & Chunking):** *Là một* quản trị viên, *tôi muốn* tải lên tài liệu chính sách dạng `.pdf`, `.txt`, `.md` để hệ thống tự động bóc tách trang và cắt đoạn (700 ký tự, 100 gối đầu), *để* cập nhật kiến thức mới mà không cần sửa mã nguồn.
16. *Acceptance:* Hỗ trợ upload tới 10MB/15 trang; sinh `KnowledgeChunk` ở trạng thái `pending`.
17. `US-09` **(Active Verification Studio):** *Là một* quản trị viên tri thức, *tôi muốn* chạy quy trình kiểm định đa tác tử đánh giá độ trung thực (Faithfulness Score $\ge 85\%$) trước khi xuất bản tài liệu, *để* đảm bảo không đưa thông tin sai lệch cho khách hàng.
18. *Acceptance:* Báo cáo trực quan `VerificationReport` với điểm trung thực và vết thực thi của 3 tác tử.
19. `US-10` **(Dual AI Provider & Cost-Guard):** *Là một* kỹ sư vận hành, *tôi muốn* chuyển đổi linh hoạt giữa Google Gemini và Local Ollama kèm cơ chế khóa Cooldown 60s khi bị giới hạn 429, *để* kiểm soát ngân sách token và thử nghiệm ngoại tuyến mượt mà.
    - *Acceptance:* Hỗ trợ `AI_PROVIDER=gemini|ollama`; Cooldown 60s tự phục hồi; có nút mở khóa thủ công.
20. `US-11` **(JWT Authentication & RBAC):** *Là một* người dùng hoặc quản trị viên, *tôi muốn* đăng nhập bảo mật bằng JWT và phân quyền nghiêm ngặt, *để* khách hàng chỉ xem được ticket của mình và bị khóa các tính năng quản trị.
    - *Acceptance:* Mã hóa HS256 chuẩn; cô lập dữ liệu khách hàng; giao diện tự động ẩn/khóa tab khi là `user`.
21. `US-12` **(SSE Streaming & Agent Confirmation):** *Là một* khách hàng tương tác với AI Agent, *tôi muốn* câu trả lời hiển thị chạy từng chữ (Streaming) và các hành động nhạy cảm (tạo ticket, đổi độ ưu tiên) phải hiện thẻ xác nhận 2 pha, *để* tôi hoàn toàn kiểm soát tài khoản.
    - *Acceptance:* Streaming SSE qua `POST /chat/stream`; hiển thị Thẻ Xác Nhận Xác Thực 2 pha trước khi ghi vào DB.

2.2 Bảng Ma trận Truy vết Yêu cầu (Requirement Traceability Matrix - RTM)

Yêu Cầu (REQ)	User Story	Mã Module / Tệp Nguồn Xử Lý	Trạng Thái Kiểm Thử E2E
`REQ-01..03` (RAG Core & Retrieval)	`US-01`	`rag_service.py`, `embedding_service.py`	100% Passed (`test_rag_module.py`)
`REQ-04` (SSE Token Streaming)	`US-12`	`chat_stream.py`, `ChatView.tsx`	100% Passed (Vite build + Manual SSE)
`REQ-05..06` (Deterministic Fallback)	`US-02`	`fallback_service.py`, `seed_faq.json`	100% Passed (`EVAL-01..05`)
`REQ-07..08` (Dual DB & PostgreSQL pgvector)	`US-07`	`session.py`, `knowledge.py`, `models/`	100% Passed (`test_pgvector_and_db.py`)
`REQ-09` (JWT Auth & Role-Based Guard)	`US-11`	`deps.py`, `auth.py`, `Header.tsx`	100% Passed (`test_auth_module.py`)
`REQ-10` (Agent Tools & 2-Phase Confirmation)	`US-12`	`agent_tools.py`, `chat_actions.py`	100% Passed (`EVAL-09..10`)
`REQ-11` (Outbound PII Redaction)	`US-03`	`security.py`, `embedding_service.py`	100% Passed (`EVAL-12`)
`REQ-12..14` (Ticket SLA & Copilot Triage)	`US-04, 05`	`ticket_copilot_service.py`, `tickets.py`	100% Passed (`test_tickets_module.py`)
`REQ-15..17` (Agent Queue Sorting & Delete)	`US-06`	`tickets.py`, `AgentDashboardView.tsx`	100% Passed (`test_tickets_module.py`)
`REQ-18..20` (Workspace Persona & Upload)	`US-07, 08`	`document_service.py`, `workspaces.py`	100% Passed (`test_workspace_and_verification.py`)
`REQ-21` (24-Case AI Evaluation Benchmark)	`US-01..03`	`run_eval.py`, `eval_dataset.json`	24/24 Passed (`100.0%`)
`REQ-22` (Cost-Guard	`US-10`	`ai_usage.py`, `TelemetryPanel.tsx`	100% Passed (Vite build & Dashboard)

Yêu Cầu (REQ)	User Story	Mã Module / Tệp Nguồn Xử Lý	Trạng Thái Kiểm Thử E2E
Cooldown & Telemetry)			
`REQ-30..34` (Multi-Agent Trace & RBAC UI)	`US-09, 11`	`VerificationTrace.tsx`, `LoginSwitcher.tsx`	100% Passed (Build & Live UI)

CHƯƠNG 3: KIẾN TRÚC HỆ THỐNG & ARCHITECTURE DECISION RECORDS (ADR)

3.1 Sơ đồ Kiến trúc Tổng thể Hệ thống (System Architecture Diagram)

```
<div class="architecture-diagram">
    <div class="arch-layer arch-client">
        <div class="arch-layer-title">Lớp Trình Duyệt Frontend (React 19 + Vite 6 + Tailwind
        <div class="arch-grid">
            <div class="arch-node">1. Customer Chat (SSE Token Streaming & 2-Phase Action Car
            <div class="arch-node">2. Ticket Form (Live Client/Server SLA Validation)</div>
            <div class="arch-node">3. Agent Dashboard (Triage, Sorting, Pagination & Delete)<
            <div class="arch-node">4. Knowledge Studio (Multi-Agent Verification Trace Modal)
            <div class="arch-node">5. Telemetry Panel (Cost-Guard 429 Cooldown Reset)</div>
            <div class="arch-node">6. RBAC Role Switcher (Admin vs User Barrier)</div>
        </div>
    </div>
    <div class="arch-arrow">▼ RESTful API (JSON) & SSE Streaming (text/event-stream) with HS2
    <div class="arch-layer arch-gateway">
        <div class="arch-layer-title">Lớp FastAPI Asynchronous ASGI Gateway (Port 8000)</div>
        <div class="arch-grid">
            <div class="arch-node">JWT HS256 Guard & RBAC Validator</div>
            <div class="arch-node">Bilingual Prompt Injection Guardrail (0.05ms)</div>
            <div class="arch-node">Outbound PII Masking (redact_pii)</div>
            <div class="arch-node">APIRouters: /chat, /tickets, /agent, /knowledge, /workspac
        </div>
    </div>
    <div class="arch-arrow">▼ Asynchronous Task Dispatcher & Execution Pipeline ▼</div>
    <div class="arch-layer arch-core">
        <div class="arch-layer-title">Lớp Xử Lý AI & Nghiệp Vụ (Core Business Logic)</div>
        <div class="arch-grid">
            <div class="arch-node">RAG Hybrid Service (Cosine + BM25, Score &ge; 0.25)</div>
            <div class="arch-node">Single-Call Ticket Copilot (Pydantic v2 Schema)</div>
            <div class="arch-node">4 DB-Backed Agent Tools with PendingAction</div>
            <div class="arch-node">3-Agent Active Knowledge Verification Studio</div>
            <div class="arch-node">Cost-Guard Circuit Breaker (60s Cooldown)</div>
            <div class="arch-node">Okapi BM25 Non-AI Deterministic Fallback</div>
        </div>
    </div>
    <div class="arch-arrow">▼ Cloud / Local AI Provider & Dual-Dialect Storage Handshake ▼</d
    <div class="arch-columns">
        <div class="arch-col arch-ai">
            <div class="arch-layer-title">Lớp Nhà Cung Cấp AI (Dual-Provider)</div>
            <div class="arch-node">Cloud: Google Gemini 3.5 Flash Lite + gemini-embedding-001
            <div class="arch-node">Local: Ollama Qwen2.5 3B + nomic-embed-text</div>
        </div>
        <div class="arch-col arch-storage">
            <div class="arch-layer-title">Lớp Lưu Trữ Dữ Liệu & Vector (Dual-Dialect)</div>
            <div class="arch-node">PostgreSQL 16 + pgvector (&lt;=&gt; Cosine Distance)</div>
            <div class="arch-node">Async SQLite (aiosqlite) + In-Python Cosine Fallback</div>
            <div class="arch-node">Externalized Prompts (prompts.json) & Seed FAQs</div>
        </div>
    </div>
</div>
```

3.2 Tóm tắt 4 Quyết định Thiết kế Kiến trúc (ADR-001 .. ADR-004)

- `ADR-001` **(Dual AI Provider & Deterministic BM25 Fallback):**

- *Bối cảnh:* Các API AI đám mây thường xuyên gặp sự cố cạn kiệt hạn ngạch (HTTP 429 Rate Limit) hoặc biến động độ trễ mạng.
- *Quyết định:* Xây dựng mô hình 3 nấc: (1) Google Gemini 3.5 Flash Lite (chính, độ trễ < 1,500ms), (2) Ollama Qwen2.5 3B (local ngoại tuyến), (3) Bộ tìm kiếm từ khóa Okapi BM25 phi AI trên tập `seed_faq.json` kích hoạt tự động sau 4.0s timeout hoặc khi mạch Cost-Guard ngắt trong 60s.
- *Hệ quả:* Đảm bảo tính sẵn sàng 100% của hệ thống; loại bỏ hoàn toàn tình trạng treo hoặc trắng màn hình.
- `ADR-002` **(Dual-Engine Vector Store: PostgreSQL pgvector + SQLite Fallback):**
- *Bối cảnh:* Cần môi trường triển khai Production mạnh mẽ với tìm kiếm vector native bằng SQL, nhưng môi trường lập trình local cá nhân cần khởi động nhanh gọn không phụ thuộc Docker nặng nề.
- *Quyết định:* Định nghĩa mô hình SQLAlchemy `Vector(768).with_variant(JSON(), "sqlite")`. Khi kết nối PostgreSQL, tự động thực thi `CREATE EXTENSION IF NOT EXISTS vector` và truy vấn bằng toán tử khoảng cách cosine `⟺`. Khi chạy trên SQLite, tự động fallback tính toán khoảng cách cosine trong bộ nhớ Python.
- *Hệ quả:* Nhà phát triển có thể chạy test tức thì trên SQLite mà không cần bật Docker, trong khi Production tận dụng tối đa chỉ mục vector PostgreSQL 16.
- `ADR-003` **(Quy trình Sát Hạch Tri Thức 3 Tác Tử - Active Knowledge Verification):**
- *Bối cảnh:* Tài liệu nội bộ công ty thường chứa các mâu thuẫn chính sách, định dạng lỗi hoặc thông tin lỗi thời, nếu nạp thẳng vào RAG sẽ gây ngộ nhận cho khách hàng.
- *Quyết định:* Áp dụng quy trình kiểm định 3 tác tử tự động trước khi xuất bản (`status="published"`):
  1. *Synthetic QA Exam Agent:* Đọc tài liệu và tự động sinh ra các cặp câu hỏi - câu trả lời thử thách.
  2. *Sandbox RAG Agent:* Gửi các câu hỏi thử thách vào pipeline RAG cô lập để sinh câu trả lời thực tế.
  3. *Faithfulness Judge Agent:* Đối chiếu câu trả lời với văn bản nguồn để chấm điểm độ trung thực (Faithfulness Score). Chỉ tài liệu đạt $\ge 85\%$ mới được mở quyền xuất bản.
- `ADR-004` **(Kiến trúc Frontend SPA React 19 + Vite 6 + Tailwind CSS v4):**
- *Bối cảnh:* Cân nhắc giữa React Single Page Application (SPA) thuần túy và Fullstack Framework Next.js.
- *Quyết định:* Lựa chọn React 19 SPA kết hợp Vite 6 và Tailwind CSS v4. Giao diện được tổ chức theo cấu trúc module chặt chẽ (`features/`, `components/`, `services/api/`) và tách biệt hoàn toàn khỏi Backend FastAPI qua hợp đồng RESTful/SSE.
- *Hệ quả:* Tốc độ biên dịch cực nhanh (~2 giây), kích thước gói bundle gọn nhẹ (< 420 KB), và dễ dàng đóng gói triển khai trên Nginx tĩnh.

CHƯƠNG 4: THIẾT KẾ AI CHUYÊN SÂU & QUY TRÌNH ĐA TÁC TỬ (MULTI-AGENT SYSTEM)

4.1 Mô hình Ngôn ngữ & Embedding Chuẩn hóa

- **Primary Cloud LLM:** Google Gemini GA `gemini-3.5-flash-lite` với cấu hình ngân sách suy nghĩ tối thiểu `thinking_budget=0` và `temperature=0.2`. Lựa chọn này giúp duy trì thời gian sinh từ đầu

tiên cực nhanh (< 1,200ms) và loại bỏ hoàn toàn nguy cơ bị bãi bỏ của các bản thử nghiệm `gemini-1.5-flash`.

- **Text Embedding:** `gemini-embedding-001` (đầu ra 768 chiều), đồng nhất với mô hình nhúng cục bộ `nomic-embed-text` của Ollama, cho phép hoán đổi vector dễ dàng giữa Cloud và Local.
- **Theo dõi chi phí thời gian thực:** Giá thành được tính trực tiếp dựa trên đơn giá token thực tế: Input \$0.30 / 1M tokens, Output \$2.50 / 1M tokens.

4.2 Quản lý Prompt Tập trung & Cách ly Đa Doanh nghiệp

Toàn bộ system prompts được phiên bản hóa trong tệp JSON ngoại vi `backend/data/prompts.json` và nạp thông qua `core/prompts.py`. Hệ thống hỗ trợ đa Persona tương ứng với từng Workspace: - **Workspace 1 (Default IT Enterprise SaaS):** Giọng văn kỹ thuật, chính xác, lịch sự, tập trung vào bảo mật và SLA. - **Workspace 2 (Nha Khoa SmileCare):** Giọng văn ân cần, thấu cảm, chuyên nghiệp y khoa, diễn giải rõ biểu phí dịch vụ và quy trình chăm sóc sau điều trị. - **Workspace 3 (Điện Máy TechStore):** Giọng văn nhanh nhẹn, định hướng bán hàng và bảo hành, hướng dẫn đổi trả trong 30 ngày.

4.3 Bốn Công cụ Tác tử (Agent Tools) có Xác nhận 2 Pha (Capability B)

Hệ thống tích hợp 4 công cụ tác tử tự động trong `backend/app/services/agent_tools.py`: 1. `lookup_customer_tickets(customer_email)`: Tra cứu danh sách các phiếu hỗ trợ hiện tại của khách hàng. 2. `create_escalation_ticket(subject, description, category, priority)`: Tạo phiếu hỗ trợ mới khi sự cố vượt quá khả năng giải đáp của chatbot. 3. `update_ticket_priority_or_status(ticket_id, new_priority, new_status)`: Cập nhật thông tin phiếu theo yêu cầu của khách hàng hoặc kỹ thuật viên. 4. `search_workspace_knowledge(query, workspace_id)`: Tìm kiếm tri thức bổ sung theo ngữ cảnh chuyên biệt.

> **LƯU Ý QUAN TRỌNG:** **Cơ chế Xác nhận 2 Pha (Two-Phase Execution):** Đối với các tác tử làm biến đổi trạng thái cơ sở dữ liệu (`create_escalation_ticket` và `update_ticket_priority_or_status`), hệ thống KHÔNG tự ý thực thi ngay mà sinh ra một đối tượng `PendingAction` với mã `action_id` duy nhất và gửi về giao diện. Người dùng bấm **[Xác nhận thực thi]** thì Backend mới chính thức ghi dữ liệu, ngăn chặn hoàn toàn rủi ro thao tác nhầm lẫn do AI tự suy luận.

4.4 Định dạng Dữ liệu Đầu ra Cấu trúc bằng Pydantic v2 (Capability C)

Toàn bộ quá trình phân loại ticket của AI Copilot được ép kiểu nghiêm ngặt bằng Pydantic v2 qua schema `TicketCopilotStructuredOutput`:

```
class TicketCopilotStructuredOutput(BaseModel):
    category: str = Field(description="Normalized category: Authentication, Billing, Bug Repo
    priority: str = Field(description="Urgent, High, Medium, or Low")
    estimated_response_hours: int = Field(description="2 for Urgent, 4 for High, 8 for Medium
    ai_tags: List[str] = Field(description="1 to 4 concise tags")
    confidence: float = Field(ge=0.0, le=1.0)
    ai_draft_reply: str = Field(description="Professional, grounded draft response in Vietnam
```

CHƯƠNG 5: AN TOÀN BẢO MẬT, HÀNG RÀO CHẮN (GUARDRAILS), QUYỀN RIÊNG TƯ & DỰ PHÒNG BM25

5.1 Hàng rào chắn Tấn công Prompt Injection Song ngữ (EN/VI)

Trước khi bất kỳ câu hỏi nào được gửi vào RAG pipeline hoặc LLM, yêu cầu phải đi qua hàm tiền xử lý `check_prompt_injection` trong `backend/app/core/security.py`. Hệ thống trang bị bộ lọc biểu thức chính quy (Regex) song ngữ phát hiện các mẫu jailbreak nguy hiểm: - **Tiếng Anh:** *"ignore all previous instructions"*, *"system override"*, *"DAN mode"*, *"reveal prompt"*, *"disregard prior guidelines"*. - **Tiếng Việt:** *"bỏ qua tất cả hướng dẫn"*, *"chế độ nhà phát triển"*, *"quên các chỉ thị trước"*, *"bỏ qua ràng buộc"*, *"tiết lộ system prompt"*. Khi phát hiện, truy vấn bị từ chối ngay lập tức tại cổng API với mã lỗi HTTP 400 và payload chuẩn:

```
{
  "detail": "PROMPT_SECURITY_VIOLATION: Suspicious prompt injection pattern detected."
}
```

5.2 Xóa mờ Thông tin Định danh Cá nhân (PII Redaction Policy)

Nhằm bảo vệ quyền riêng tư người dùng theo chuẩn bảo mật dữ liệu, hàm `redact_pii(text)` tự động quét và thay thế các thông tin nhạy cảm trước khi văn bản được đưa vào mô hình nhúng (Embedding) hoặc chuyển đến Cloud LLM: - **Email:** `nguyen.an@company.vn` $\to$ `[REDACTED_EMAIL]` - **Số điện thoại:** `0912345678` $\to$ `[REDACTED_PHONE]` - **Số Căn cước Công dân / CMND:** `079098765432` $\to$ `[REDACTED_ID]`

5.3 Ngưỡng Lọc Phù hợp & Dự phòng Okapi BM25 Circuit Breaker

- **Bộ lọc điểm tương đồng:** Hệ thống áp dụng ngưỡng sàn `RAG_MIN_SIMILARITY_SCORE = 0.25` và điểm lexical Okapi BM25 ≥ 1.2. Các câu hỏi nằm ngoài phạm vi tri thức (out-of-scope như thời tiết, nấu ăn, giá tiền mã hóa) sẽ tự động kích hoạt cờ `escalation_recommended=True` và điểm tin cậy `confidence < 0.65`, từ chối trả lời ảo giác và đề xuất kết nối nhân viên hỗ trợ.
- **Cost-Guard Circuit Breaker:** Khi Google Gemini trả về mã lỗi HTTP 429 (`ResourceExhausted`), hệ thống tự động khóa cổng gọi Cloud trong 60 giây (`FREE_TIER_RATE_LIMIT_COOLDOWN`), chuyển toàn bộ lưu lượng sang bộ so khớp BM25 cục bộ, bảo vệ hệ thống khỏi tình trạng treo hoặc sập dịch vụ. Quản trị viên có thể mở khóa mạch ngắt bất kỳ lúc nào bằng nút **[Mở Khóa Cooldown 429]** trên giao diện Telemetry.

CHƯƠNG 6: KẾT QUẢ THỰC NGHIỆM BỘ 24 TEST CASES, CHI PHÍ TOKEN & ĐỘ TRỄ

6.1 Phương pháp Đánh giá Thực nghiệm

Toàn bộ năng lực của hệ thống được kiểm chứng qua bộ dữ liệu 24 kịch bản kiểm thử độc lập `backend/data/eval_dataset.json` thông qua script tự động `backend/scripts/run_eval.py`. - **Nhóm Normal (10 cases):** Kiểm tra khả năng trích xuất tri thức chính xác kèm trích dẫn, gọi công cụ tra cứu và sinh thẻ xác nhận hành động. - **Nhóm Edge (6 cases):** Kiểm tra tiếng Việt không dấu, truy vấn chứa thông tin PII, câu hỏi đa sự kiện và ép kiểu Pydantic v2 Structured Output. - **Nhóm Should-**

Refuse (8 cases): Kiểm tra chặn tấn công Prompt Injection song ngữ (EN/VI) và từ chối câu hỏi ngoài phạm vi tài liệu (Out-of-Scope).

6.2 Bảng So Sánh Hiệu Quả Trước và Sau Tối Ưu (Before vs. After)

Phân Loại Kịch Bản	Số Ca Kiểm Thử	Tỷ Lệ Đạt Ban Đầu (Baseline)	Tỷ Lệ Đạt Sau Tối Ưu (Upgraded)	Mức Độ Cải Thiện (Δ)
Normal (In-Scope, RAG & Tools)	10	5/10 (`50.0%`)	**10/10 (`100.0%`)**	`+50.0%`
Edge (No-Diacritics, PII, Schema)	6	1/6 (`16.7%`)	**6/6 (`100.0%`)**	`+83.3%`
Should-Refuse (Prompt Injection & Out-of-Scope)	8	2/8 (`25.0%`)	**8/8 (`100.0%`)**	`+75.0%`
TỔNG HỢP TOÀN BỘ BENCHMARK	**24**	**8/24 (`33.3%`)**	**24/24 (`100.0%`)**	`+66.7%` **(`+16` test cases)**

6.3 Phân Tích Độ Trễ & Chi Phí Token

- **Thời gian phản hồi câu hỏi thông thường (Live Gemini 3.5 Flash Lite):** Trung bình **920ms – 1,350ms** (đạt mục tiêu SLA đề ra $< 1,500ms$).
- **Thời gian phản hồi chế độ dự phòng BM25 (Deterministic Fallback):** Cực nhanh, trung bình **12ms – 18ms**.
- **Thời gian chặn Prompt Injection tại Gateway:** Chỉ mất **0.03ms – 0.06ms** (xử lý ngay trước khi gọi database hoặc LLM).
- **Chi phí vận hành trung bình:** Xấp xỉ **\$0.00018 – \$0.00035 USD** trên mỗi lượt trao đổi hội thoại hoàn chỉnh.

CHƯƠNG 7: HẠN CHẾ HIỆN TẠI & ĐỊNH HƯỚNG PHÁT TRIỂN

7.1 Hạn chế hiện tại của Hệ thống

1. **Phân tích tài liệu có bảng biểu phức tạp:** Trình bóc tách văn bản PDF hiện tại (`pypdf`) chủ yếu xử lý định dạng văn bản thô, chưa nhận diện tối ưu các bảng biểu ma trận phức tạp hoặc biểu đồ hình ảnh (cần mô hình Vision / Multimodal OCR).
2. **Ngôn ngữ mở rộng:** Mặc dù hệ thống hỗ trợ tiếng Việt và tiếng Anh rất tốt, các ngôn ngữ khác (tiếng Nhật, tiếng Hàn, tiếng Trung) chưa được tối ưu hóa sâu trong bộ từ điển BM25.
3. **Cơ chế phân tải phân tán (Distributed Queue):** Tác vụ nhúng tài liệu lớn hiện đang xử lý qua hàng đợi async in-memory, cần tích hợp Celery + Redis khi số lượng tài liệu đồng thời tăng vọt.

7.2 Định hướng phát triển tương lai

- **Tích hợp Multimodal Document Processing:** Sử dụng Gemini 2.0 Flash Vision để đọc hiểu trực tiếp sơ đồ kỹ thuật và biểu đồ trong tài liệu PDF.
- **Reranker Chuyên sâu:** Bổ sung mô hình Cohere Rerank hoặc BGE-Reranker sau giai đoạn truy xuất để tăng thêm 15–20% độ chính xác cho các câu hỏi dài.

- **Kênh Tương Tác Đa Nền Tảng (Omnichannel):** Mở rộng kết nối webhook tự động tới Telegram Bot, Zalo Official Account và Slack Channel của doanh nghiệp.

Báo cáo được hoàn thiện và xác thực đồng bộ với toàn bộ mã nguồn của dự án SmartDesk AI.